

Implementar el desarrollo de servicios en aplicaciones web

EJE 3

Integrantes:

Fabio Andrés Bejarano Casas

Navin Andrés Balmaceda Monzón

Juan López García

PROFESOR:

DEIVYS MORALES URIBE

Universidad Área Andina

MAYO- 2026

En la actualidad, el desarrollo web ha cambiado mucho debido a la necesidad de crear aplicaciones más dinámicas, seguras y conectadas. Las organizaciones necesitan sistemas que puedan compartir información en tiempo real entre diferentes plataformas, lenguajes de programación y bases de datos. Esto permite optimizar procesos y mejorar la experiencia de los usuarios. Por esta razón, los servicios web se han convertido en herramientas clave en el desarrollo de software moderno. Facilitan la comunicación entre aplicaciones mediante protocolos y estándares como HTTP, XML, JSON, SOAP y REST.

En este contexto, la arquitectura orientada a servicios (SOA) ayuda a estructurar aplicaciones distribuidas en módulos independientes. Esto facilita la integración de funciones y el intercambio de datos entre sistemas.

La aplicación implementa servicios web REST para realizar operaciones de consulta, registro, actualización y eliminación de datos. Esto permite que el cliente y el servidor se comuniquen de manera eficiente. Para su construcción, se utilizaron herramientas de desarrollo web, bibliotecas y tecnologías enfocadas en mejorar la funcionalidad, el diseño y el rendimiento de la aplicación. Desarrollar esta actividad permitió aplicar los conocimientos adquiridos sobre servicios web, arquitectura SOA, métodos de negocio y separación de capas. Esto fortaleció las habilidades relacionadas con el desarrollo de aplicaciones web modernas y la integración de servicios entre sistemas.

OBJETIVO GENERAL

Desarrollar una aplicación web orientada a la gestión clínica, implementando servicios web tipo REST y aplicando una arquitectura basada en capas y principios SOA, con el fin de integrar componentes que permitan la comunicación entre aplicaciones web y la administración eficiente de la información.

OBJETIVOS ESPECÍFICOS

El proyecto consiste en el desarrollo de una aplicación web clínica que permite administrar información relacionada con pacientes, citas médicas y servicios de salud, utilizando tecnologías web modernas y una arquitectura basada en capas.

La aplicación implementa servicios web REST para facilitar la comunicación entre el frontend y el backend, permitiendo realizar operaciones CRUD (crear, consultar, actualizar y eliminar información). Además, se aplican conceptos de arquitectura orientada a servicios (SOA), separación de capas y lógica de negocio vistos en el referente de pensamiento del eje 3.

Aplicación de arquitectura SOA: la arquitectura orientada a servicios (SOA) permite integrar servicios distribuidos y mejorar la comunicación entre sistemas. Según el referente, SOA facilita la exposición e invocación de servicios mediante estándares como XML, HTTP, SOAP y REST. En la aplicación clínica se utilizaron principios SOA para separar funcionalidades independientes y permitir que diferentes módulos puedan comunicarse de manera organizada.

1. Inicialización de Flask

El código crea una aplicación web utilizando Flask:

```
dashboard_app.py > dashboard
1  # dashboard_app.py
2  from flask import Flask, render_template, jsonify
3  import requests
4
5  app = Flask(__name__)
6
```

Esto permite levantar un servidor web para mostrar páginas y manejar rutas. Definimos la dirección donde se está ejecutando la api de la clínica, como se observa en la imagen está corriendo de manera local y a través del puerto 5000

```
7  # URL donde vive actualmente tu API (el proyecto de la clínica)
8  URL_API_CLINICA = "http://localhost:5000"
9
```

Consumo de la API REST

El sistema realiza peticiones HTTP GET para obtener información de médicos, pacientes y citas:

```
10 @app.route('/')
11 def dashboard():
12     # 1. Consumimos los Médicos
13     res_medicos = requests.get(f"{URL_API_CLINICA}/medicos")
14     medicos_data = res_medicos.json() if res_medicos.status_code == 200 else []
15
16     # 2. Consumimos los Pacientes
17     res_pacientes = requests.get(f"{URL_API_CLINICA}/pacientes")
18     pacientes_data = res_pacientes.json() if res_pacientes.status_code == 200 else []
19
20     # 3. Consumimos las citas
21     res_citas = requests.get(f"{URL_API_CLINICA}/citas")
22     citas_data = res_citas.json() if res_citas.status_code == 200 else []
23
```

Aquí la aplicación se conecta a una API externa o local para traer datos en formato JSON.

El request realiza solicitudes a la API para consultar médicos, pacientes y citas. Si la respuesta es correcta (código 200), los datos se convierten a formato JSON, en caso de error, se devuelve una lista vacía para evitar fallos en el dashboard

Conversión de datos JSON

Después de obtener la respuesta, el sistema transforma los datos:

```
23  
24     # 4. Le pasamos AMBAS listas al template HTML  
25     return render_template(  
26         'dashboard.html',  
27         medicos=medicos_data,  
28         pacientes=pacientes_data,  
29         citas=citas_data  
30     )
```

Esto convierte la respuesta JSON en estructuras de Python para poder trabajar con ellas.

Activa el modo desarrollo. (debug=True) Sirve para: detectar errores, recargar automáticamente cambios, mostrar mensajes de depuración.

```
30     )  
31     if __name__ == '__main__':  
32         # Corremos el dashboard en un puerto diferente (5001) para que no choque con la clínica  
33         app.run(port=5001, debug=True)
```

Usar el puerto 5001 para evitar conflictos con otra aplicación.

Tarjetas MEDICOS

La siguiente sección crea una tarjeta visual para mostrar estadísticas.

card crea una tarjeta

bg-primary color azul

text-white texto blanco

text-center centra contenido

shadow-sm agrega sombra

```
<!-- MÉDICOS -->
<div class="col-md-4">
  <div class="card stat-card bg-primary text-white text-center shadow-sm">
    <div class="card-body">
      <h4 class="card-title">
        | 🧑‍⚕️ Médicos Totales
      </h4>

      <div class="display-number">
        | {{ medicos|length }}
      </div>
    </div>
  </div>
</div>
```

Tarjetas PACIENTES

```
<!-- PACIENTES -->
<div class="col-md-4">
  <div class="card stat-card bg-success text-white text-center shadow-sm">
    <div class="card-body">
      <h4 class="card-title">
        🧑 Pacientes Registrados
      </h4>

      <div class="display-number">
        {{ pacientes|length }}
      </div>
    </div>
  </div>
</div>
```

Tarjetas CITAS

```
<!-- CITAS -->
<div class="col-md-4">
  <div class="card stat-card bg-dark text-white text-center shadow-sm">
    <div class="card-body">
      <h4 class="card-title">
        📅 Citas Agendadas
      </h4>

      <div class="display-number">
        {{ citas|length }}
      </div>
    </div>
  </div>
</div>
```

Crear las tablas

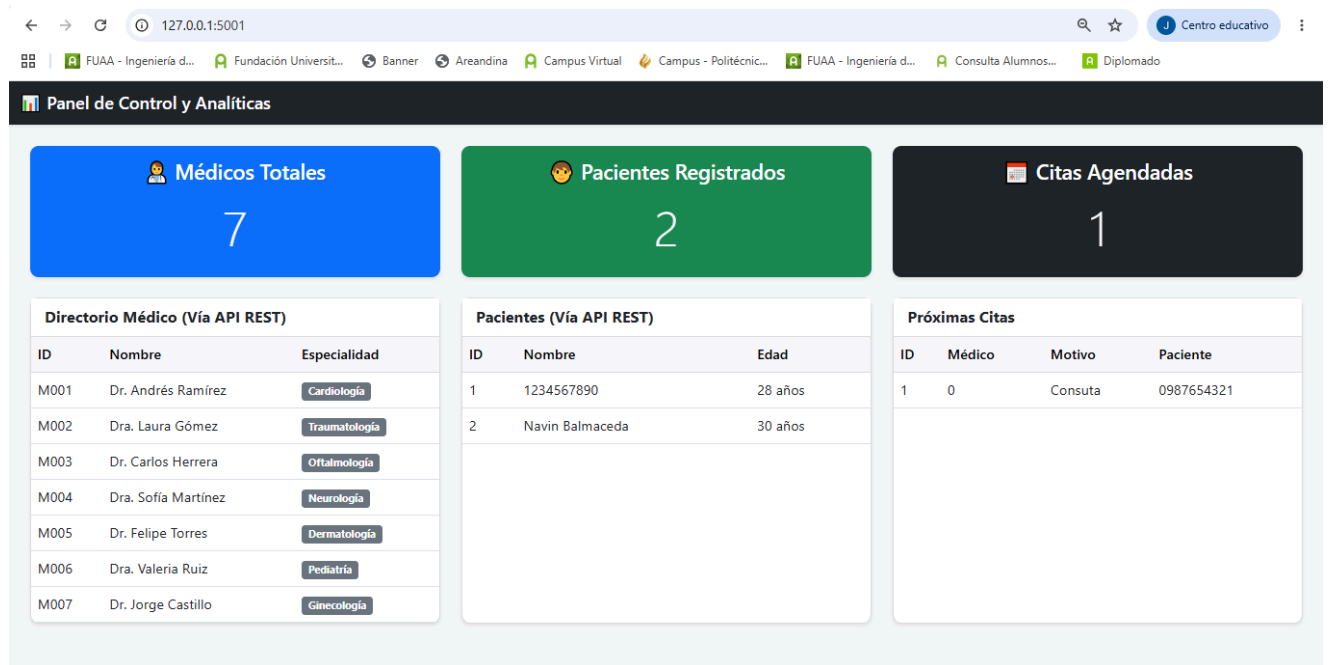
Esa parte del código se encarga de crear una tabla dinámica que muestra los médicos obtenidos desde la API REST.

```
113 <!-- MÉDICOS -->
114 <div class="col-md-4 d-flex">
115   <div class="card shadow-sm w-100">
116     <div class="card-header bg-white fw-bold">
117       Directorio Médico (Via API REST)
118     </div>
119     <div class="card-body p-0">
120       <table class="table table-hover mb-0">
121         <thead class="table-light">
122           <tr>
123             <th>ID</th>
124             <th>Nombre</th>
125             <th>Especialidad</th>
126           </tr>
127         </thead>
128         <tbody>
129           {% for m in medicos %}
130           <tr>
131             <td>{{ m.id }}</td>
132             <td>{{ m.nombre }}</td>
133             <td>
134               <span class="badge bg-secondary">
135                 {{ m.especialidad }}
136               </span>
137             </td>
138           </tr>
139           {% else %}
140           <tr>
141             <td colspan="3"
142               class="text-center text-muted py-3">
143               No hay médicos / API no responde
144             </td>
145           </tr>
146           {% endfor %}
147         </tbody>
148       </table>
149     </div>
150   </div>
151 </div>
```


Consumo de API Dashboard

Estas expresiones pertenecen al motor de plantillas de Flask y sirven para contar registros y mostrarlos en pantalla.

Ejemplo: Médicos totales, Pacientes registrados, Citas agendadas



Registro de paciente nuevo en el sistema

Las imágenes muestran formularios para: Registrar pacientes, Solicitar citas médicas El usuario ingresa datos y estos se envían a la API para almacenarlos en la base de datos.

| Registro de Paciente



Formulario de Registro de Paciente:

- Numero de identificación: 3214567890
- Nombre Completo: Yesica Fuentes
- Edad: 22
- Botón: Registrarse

Solicitud de cita nueva

| Solicitar Cita Médica



Formulario de Solicitar Cita Médica:

- Numero de identificación: 3214567890
- Médico Dependiente: Dra. Sofía Martínez — Neurología
- Fecha de la Cita: 27/05/2026
- Motivo de la Consulta: Revisión
- Botón: Agendar Cita

Se observa que los registros en el dashboard se actualizan, lo que indica una correcta comunicación de nuestra clínica con la API del dashboard

👤 Pacientes Registrados			📅 Citas Agendadas			
3			2			
Pacientes (Vía API REST)			Próximas Citas			
ID	Nombre	Edad	ID	Médico	Motivo	Paciente
1	1234567890	28 años	1	0	Consulta	0987654321
2	Navin Balmaceda	30 años	2	0	Revisión	3214567890
3	Yesica Fuentes	22 años				

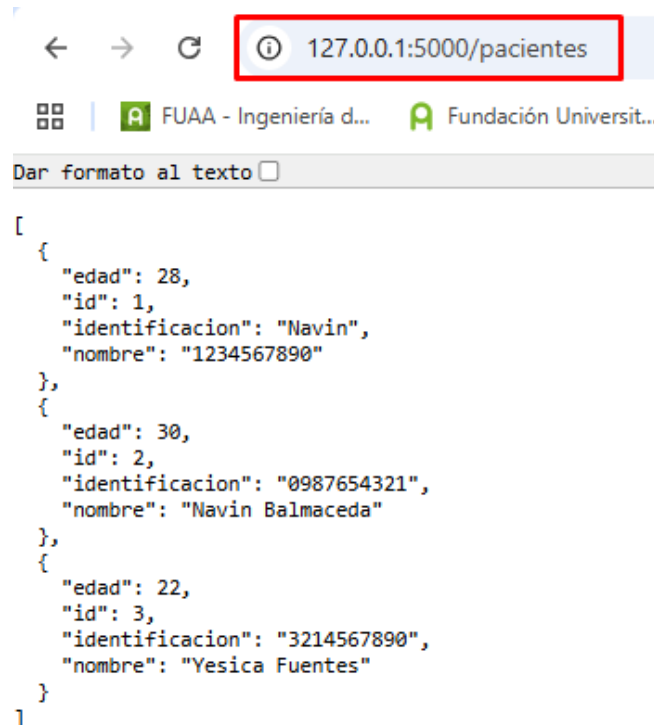
Se observa correcta respuesta y almacenamiento del json



```
[
  {
    "fecha": "Tue, 19 May 2026 00:00:00 GMT",
    "id": 1,
    "medico_id": 0,
    "motivo": "Consuta",
    "paciente_id": "0987654321"
  },
  {
    "fecha": "Wed, 27 May 2026 00:00:00 GMT",
    "id": 2,
    "medico_id": 0,
    "motivo": "Revisi\u00f3n",
    "paciente_id": "3214567890"
  }
]
```

Esto confirma que el backend y el frontend se comunican correctamente mediante servicios

REST



```
[
  {
    "edad": 28,
    "id": 1,
    "identificacion": "Navin",
    "nombre": "1234567890"
  },
  {
    "edad": 30,
    "id": 2,
    "identificacion": "0987654321",
    "nombre": "Navin Balmaceda"
  },
  {
    "edad": 22,
    "id": 3,
    "identificacion": "3214567890",
    "nombre": "Yesica Fuentes"
  }
]
```

Los servicios web REST permiten mejorar la comunicación entre aplicaciones web mediante el uso de protocolos HTTP y formatos ligeros como JSON.

La arquitectura SOA facilita la integración de diferentes componentes y servicios, permitiendo aplicaciones más escalables y organizadas.

La separación en capas (presentación, negocio y datos) mejora la estructura del software y facilita su mantenimiento.

Los servicios SOAP siguen siendo importantes en entornos empresariales debido a sus estándares de seguridad y confiabilidad.

El desarrollo del dashboard clínico permitió aplicar los conocimientos adquiridos sobre arquitectura web, lógica de negocio y consumo de servicios.

VIDEO EXPLICATIVO:

https://youtu.be/0b1h9cRqI_4

BIBLIOGRAFÍA

REST - Glosario de MDN Web Docs | MDN. (2023, 13 noviembre).

https://developer.mozilla.org/es/docs/Glossary/REST?utm_source

Zamora, D. (2023, 31 octubre). Cómo crear una página web (responsive) en HTML

y CSS. DropCoding: Aprende Diseño y Desarrollo Web Desde Cero.